

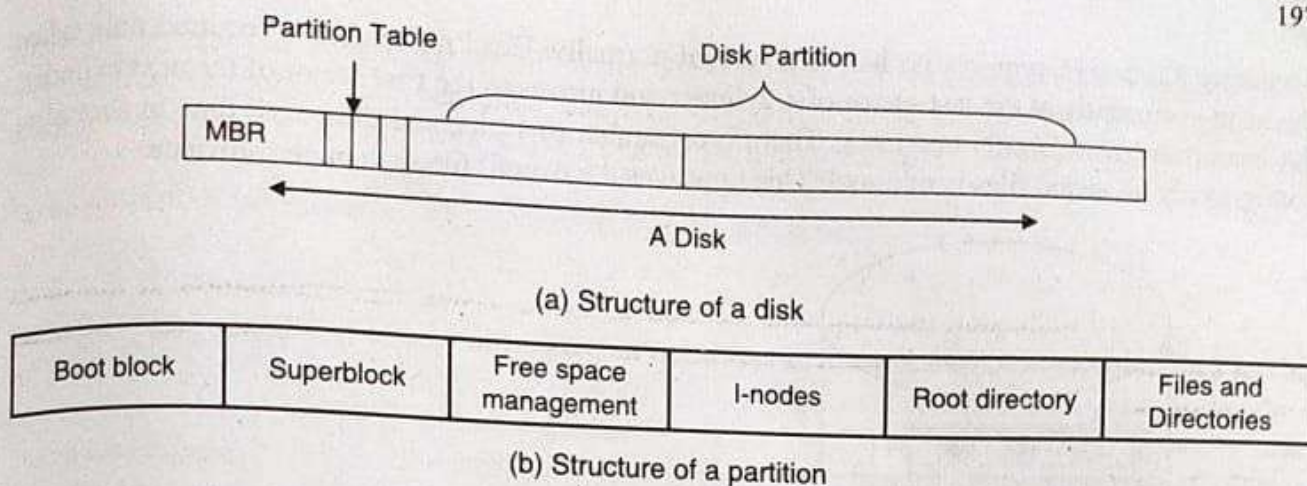


Figure 12.2: File System Layout

The in-memory structure helps in improving the performance of the file system. The in-memory structures include:

- **In-memory partition table:** It stores information about each mounted partition.
- **In-memory directory structure:** It stores the information about the recently accessed directories.
- **System-wide open-file table:** It contains a copy of the FCB of each open file as also keeps count of the number of processes that have a file open.
- **Per-process open-file table:** It contains a pointer to the corresponding entry in the system-wide open-file table along with some related information.

When the `open` call passes a file name to the file system, the directory structure is searched for the given file name. When the file is found, its FCB is copied into the system-wide open-file table, and the count is incremented.

After updating the system-wide open-file table, an entry is made in the per-process open-file table. This entry includes a pointer to the appropriate entry in the system-wide open-file table, a pointer to the position in the file where the next read or write will occur, and the mode in which the file is open. The `open` call returns a pointer to the appropriate entry in the per-process file system table. This pointer is used to perform all the operations as long as the file is open. It means that until a file is closed, all the operations are carried out on the open-file table.

When a process closes the file, the corresponding entry from the per-process open-file table is removed and the system-wide entry's open count is decremented. The value in the count indicates the number of users who have opened the file currently. Thus, when this value becomes 0, the updated file information is copied back to the disk-based structures and the entry is removed from the system-wide open-file table.

12.3 IMPLEMENTING FILES

A major issue regarding implementation of files on the secondary storage such as disk is to keep track of which disk blocks have been allocated to which files. Some widely used methods for allocating disk space to files are *contiguous*, *linked list*, *linked list using index*, and *i-nodes*. For discussing these different allocation methods, a file is considered to be a sequence of blocks and all I/O operations on a disk occur in terms of blocks.

12.3.1 Contiguous Allocation

In contiguous allocation, each file is allocated contiguous blocks on the disk, that is, one after the other (see Figure 12.3). Assuming only one job is accessing the disk, once the first block, say *b*, is accessed,

accessing block $b+1$ requires no head movement normally. Head movement is required only when the head is currently at the last sector of a cylinder and moves to the first sector of the next cylinder; the head movement is only one track. Therefore, number of seeks, and thus, seek time in accessing contiguously allocated files is minimal. This improves the overall file system performance.

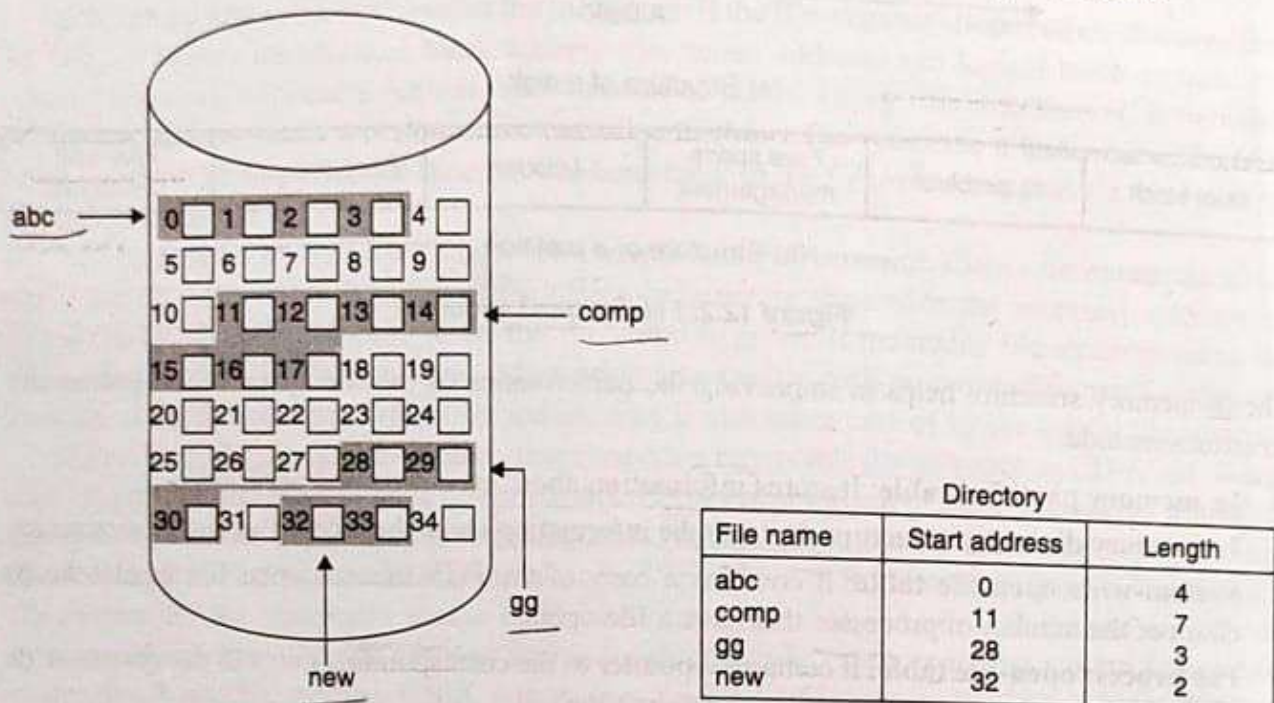


Figure 12.3: Example of a Contiguous Allocation

It is relatively simple to implement the file system using contiguous allocation method. The directory entry for each file contains the file name, the disk address of the first block, and the total size of the file.

Adv. Contiguous allocation supports both sequential and direct access to a file. For sequential access, the file system remembers the disk address of the last block referenced, and when required, reads the next block. For direct access to block b of a file that starts at location L , the block $L+b$ can be accessed immediately.

Dis Adv. Contiguous allocation has a significant problem, that of external fragmentation. Initially, the disk is free, and each new file can be allocated contiguous blocks starting from the block where the previous file ends. When a file is deleted, it leaves behind some free blocks in the disk. This is not a problem as long as we have contiguous blocks to allocate to a new file at the end of disk. However, with time, the disk will become full, and at that time the free blocks are fragmented throughout the disk. One solution to this problem is **compaction**, which involves moving the blocks on the disk to make all free space into one contiguous space. Compaction is expensive in terms of time as it may take hours to compact a large hard disk that uses contiguous allocation. Moreover, normal operations are generally suspended during compaction.

An alternative to expensive compaction is to reuse the space. For this, we need to maintain a list of holes (an unallocated segment of contiguous blocks). In addition, we must know the final size of a file at the time of its creation so that a sufficiently large hole can be allocated to it. However, determining the file size in advance is generally difficult and allocating either too little or too much space to a file is a problem. If we allocate more space than that it needs, we will end up wasting costly memory. On the other hand, if we allocate too little space, we will not be able to extend the file, since the blocks on either side of the file may be allocated to some other files. One possibility exists: we can extend the space by terminating the user program, when the user can restart it with more

space. However, restarting the user program repeatedly may prove costly. Alternatively, the system may find the larger hole, copy the contents of the file to the new space, and release the previous space. This can be done repeatedly as long as the space required is available contiguously in the disk. Moreover, the user program need not be restarted, with the user remaining oblivious about these developments. However, the task is again time consuming.

12.3.2 Linked List Allocation

The file size generally tends to change (grow and shrink) over time. The contiguous allocation of such files results in several problems. Linked list allocation method overcomes all the problems associated with the contiguous allocation method.

In the linked list allocation method, each file is stored as a linked list of disk blocks. The disk blocks are generally scattered throughout the disk, with each disk block storing the address of the next block. The directory entry contains the file name and the address of the first and last blocks of the file (see Figure 12.4).

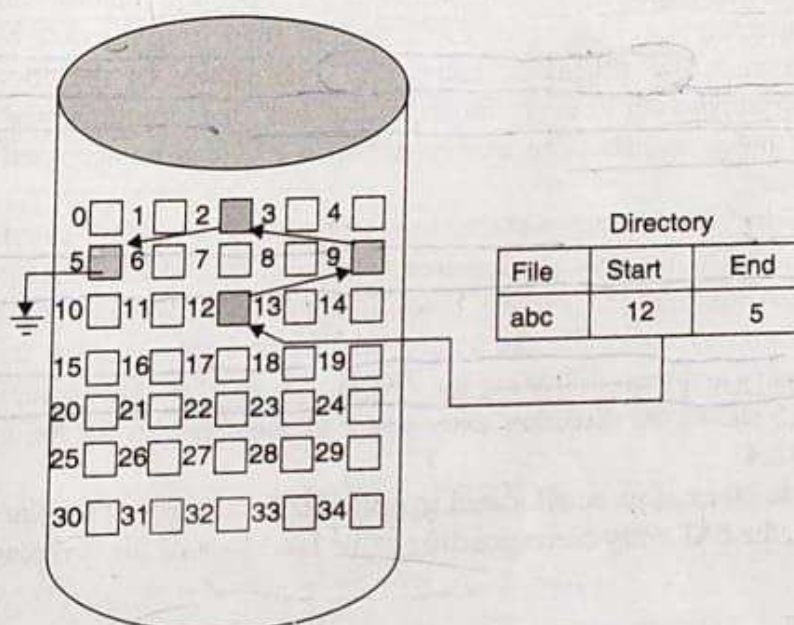


Figure 12.4: An Example of Linked List Allocation

This figure shows the linked list allocation for a file. A total of four disk blocks are allocated to the file. The directory entry indicates that the file starts at block 12. It then continues at block 9, block 2, and finally ends at block 5.

The simplicity and straightforwardness of this method makes it easy to implement. The linked list allocation results in optimum utilization of disk space, since even a single free block between the used blocks can be linked and allocated to a file. This method does not give rise to the problem of external fragmentation, so compaction is never required.

The main disadvantages of using linked list allocation are slow access speed, disk space utilization by pointers, and low-reliability of the system. As this method provides only sequential access to files, therefore, to find out the n^{th} block of a file, the search starts at the beginning of the file and follows the pointer until the n^{th} block is found. This means that there will be high average turn around time for very large files.

In linked list allocation, maintaining pointers in each block requires some disk space. The total disk space required by all the pointers in a file becomes substantial, which results in more space required by each file. The space required to store pointers can otherwise be used to store the

information. To overcome this problem, contiguous blocks are grouped together as a **cluster**, and allocation to files takes place as clusters rather than blocks. Clusters allocated to a file are then linked together. Having a pointer per cluster rather than per block reduces the total space needed by all the pointers. This approach also improves the disk throughput, since fewer disk seeks are required. However, this approach may increase internal fragmentation, because having a partially full cluster wastes more space than having a partially full block.

Moreover, linked list allocation is not very reliable. Since disk blocks are linked together by pointers, a single damaged pointer may prevent us from accessing the file blocks that follow the damaged link. Some operating systems deal with this problem by creating special files for storing redundant copies of pointers. One copy of the file is placed in main memory to provide faster access to disk blocks. Other files of redundant pointers help in safer recovery.

12.3.3 Linked List Allocation using Index

In linked allocation method, problems arise because of storing pointer information within the disk blocks. To avoid these problems, a variation of linked list allocation method can be used. The idea behind this variant method is to keep pointer information apart from the disk block and store it in a separate table known as **file allocation table (FAT)** maintained by the operating system. FAT contains an entry corresponding to every block on the disk. It is implemented as an array and is indexed by the disk block number. The entries stored in FAT can be accessed in much the same manner as a linked list.

The directory entry of a file stores the address of first disk block allocated to the file; and the FAT entry corresponding to this disk block points to the next disk block allocated to that file and so on. The FAT entry corresponding to last disk block allocated to the file contains an end-of-file value (for example, -1). Thus, the entire file can be accessed by first obtaining the starting block address from the directory entry and then following the FAT entries until an entry indicating the end-of-file is found. Figure 12.5 shows the directory entry and FAT table entries for the linked list allocation depicted in Figure 12.4.

Whenever a new block is to be allocated to a file, FAT is searched for the first free block. On finding a free block, the FAT entry corresponding to the last block of file is replaced with the address

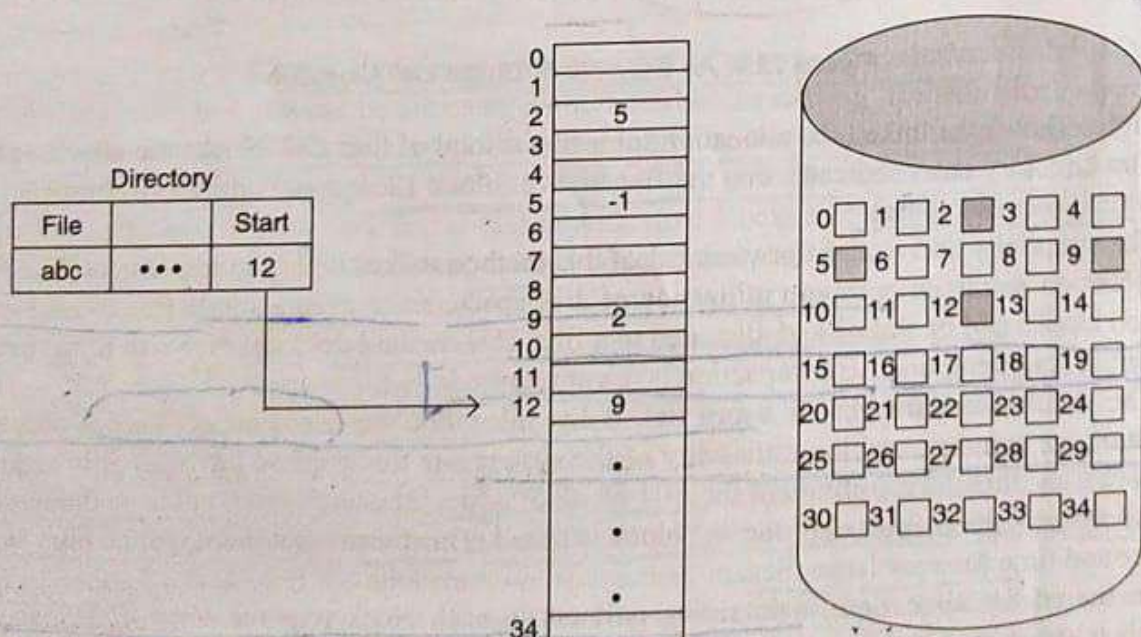


Figure 12.5: An Example of Linked List Allocation using Index

of this free block. In addition, the FAT entry of the newly allocated block is replaced with end-of-file value as now this is the last block allocated to the file.

FAT scheme is simple and efficient. However, it results in significant number of disk accesses and in turn, poor performance. That is because FAT resides on the disk and has to be accessed every time we need to obtain the address of the next disk block. Thus, to make this allocation scheme work, FAT must be kept in main memory while the file is being processed. But as the size of FAT is proportional to the size of disk, a huge amount of space is to be kept reserved in main memory. The situation becomes even worse when the disk size grows which, in turn, increases the table size.

Note: Some operating systems such as MS-DOS and Windows employ File Allocation Table scheme.

12.3.4 I-nodes

In this allocation method, each file is associated with a data structure known as **i-node (index-node)**. The i-node of a specific file contains its attributes and the disk addresses of the blocks allocated to it. This implies that the blocks of any file can be found by just having the i-node of that file in the main memory. This proves a major advantage over FAT scheme as at any instant only i-nodes of the open files need to be in main memory thereby occupying much smaller space in main memory as compared to FAT scheme. Moreover, the amount of space reserved in main memory increases as the number of open files increase, and is not linked to the size of disk.

Each i-node is capable of storing only a limited number of disk block addresses; therefore a problem arises when the size of a file increases beyond the i-node's limit. As a solution to this, indirect pointers are used in i-nodes. Initial entries of an i-node store address of the disk blocks containing the file data. The last entry of an i-node contains address of a disk block, known as **single indirect block**, which contains pointers to the additional blocks allocated to the file. (Figure 12.6).

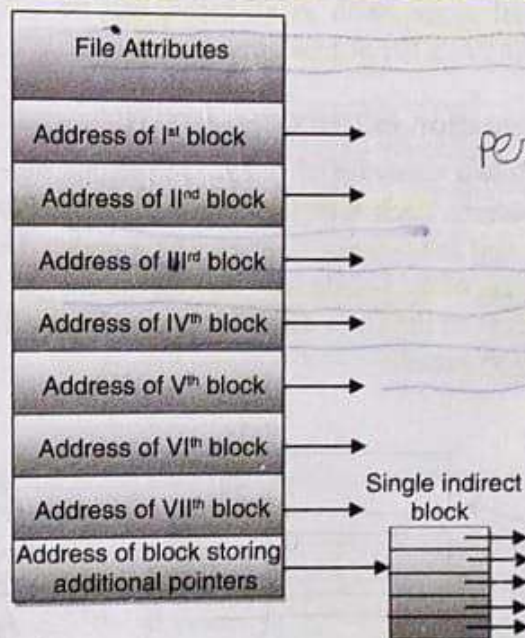


Figure 12.6: Example of an I-node

Note that to support even larger files, two or more entries in i-node can also be made to store addresses of disk blocks that contain pointers to the data blocks (disk blocks containing the file data). Sometimes, using multiple single-indirect blocks is not enough, so some operating systems (for example, UNIX) allow using double indirect blocks, to support larger file sizes. An i-node entry stores the address of a **double indirect block**, which contains the address of a disk block storing a

list of single indirect blocks. Each of these single indirect blocks contains pointers to data blocks. In case double indirect block is also not enough, triple indirect block can be used.

12.4 IMPLEMENTING DIRECTORIES

The efficiency, performance, and reliability of a file system are directly related to the directory-management and directory-allocation algorithms selected for a file system. Here, we will discuss the directory implementation in MS-DOS and UNIX.

12.4.1 Directory Implementation in MS-DOS

MS-DOS uses the linear list method for implementing directories. In this method, a directory is organized as a collection of fixed size entries, where each entry contains a (fixed-length) file name, a fixed structure to store the file attributes, and pointers to the data blocks (see Figure 12.7). All attributes of the file are stored at one place as a single directory entry and linear search is used to find a directory entry from the list of entries. It means to search out an entry in the list of entries. Each entry (starting from the first entry in the directory) is examined one by one until the desired entry is found. This is simple to program; however, the search becomes very slow with extremely large directories, which is a major disadvantage of this method.

When the user sends a request to create a new file, the directory is searched to check whether any other file has the same name. If no other file has the same name, the memory will be allocated and an entry for the same would be added at the end of the directory. To delete a file, the directory is searched for the file name and if the file is found, the space allocated to it is released. The delete operation results in free space that can be reused. To reuse this space, it can be marked with a used-unused bit, and a special name, such as all-zeros, can be assigned to it, or it can be linked to a list of free directory entries.

Directory

Itles1	attributes
language	attributes
magazine	attributes
sports	attributes
computer	attributes

12.4.2 Directory Implementation in UNIX

UNIX operating systems follow a variation of linear list method for recording the file information. Each directory entry consists of two fields: the file name and the i-node number. The i-node number contains the disk address of the i-node structure that stores the file attributes and the address of the file's data blocks (see Figure 12.8). With this approach, the size of the directory entry is very small.

Figure 12.7: Example of a Linear List Directory Entry in MS-DOS

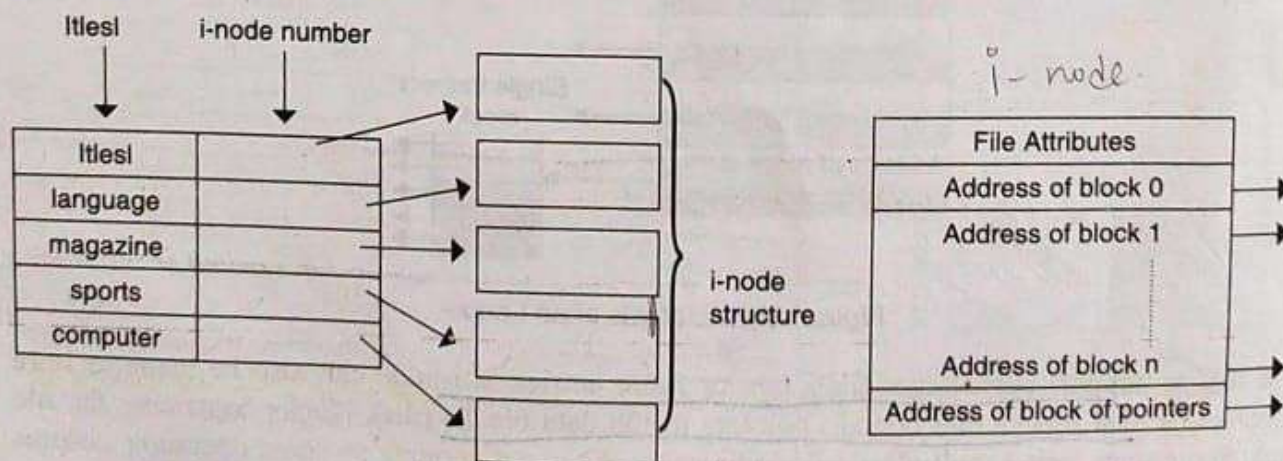


Figure 12.8: Example of a Linear List Directory Entry in UNIX

accessing block $b+1$ requires no head movement normally. Head movement is required only when the head is currently at the last sector of a cylinder and moves to the first sector of the next cylinder; the head movement is only one track. Therefore, number of seeks, and thus, seek time in accessing contiguously allocated files is minimal. This improves the overall file system performance.

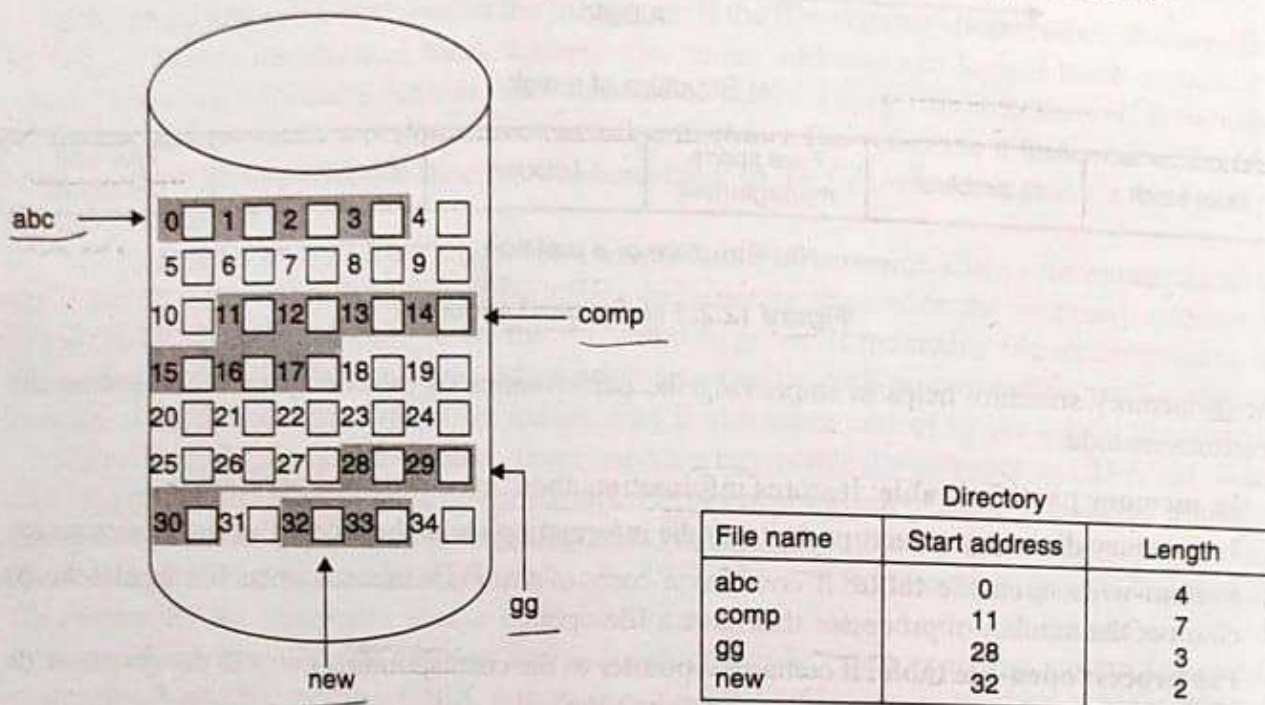


Figure 12.3: Example of a Contiguous Allocation

It is relatively simple to implement the file system using contiguous allocation method. The directory entry for each file contains the file name, the disk address of the first block, and the total size of the file.

Adv. Contiguous allocation supports both sequential and direct access to a file. For sequential access, the file system remembers the disk address of the last block referenced, and when required, reads the next block. For direct access to block b of a file that starts at location L , the block $L+b$ can be accessed immediately.

Dis Adv. Contiguous allocation has a significant problem, that of external fragmentation. Initially, the disk is free, and each new file can be allocated contiguous blocks starting from the block where the previous file ends. When a file is deleted, it leaves behind some free blocks in the disk. This is not a problem as long as we have contiguous blocks to allocate to a new file at the end of disk. However, with time, the disk will become full, and at that time the free blocks are fragmented throughout the disk. One solution to this problem is **compaction**, which involves moving the blocks on the disk to make all free space into one contiguous space. Compaction is expensive in terms of time as it may take hours to compact a large hard disk that uses contiguous allocation. Moreover, normal operations are generally suspended during compaction.

An alternative to expensive compaction is to reuse the space. For this, we need to maintain a list of holes (an unallocated segment of contiguous blocks). In addition, we must know the final size of a file at the time of its creation so that a sufficiently large hole can be allocated to it. However, determining the file size in advance is generally difficult and allocating either too little or too much space to a file is a problem. If we allocate more space than that it needs, we will end up wasting costly memory. On the other hand, if we allocate too little space, we will not be able to extend the file, since the blocks on either side of the file may be allocated to some other files. One possibility exists: we can extend the space by terminating the user program, when the user can restart it with more

space. However, restarting the user program repeatedly may prove costly. Alternatively, the system may find the larger hole, copy the contents of the file to the new space, and release the previous space. This can be done repeatedly as long as the space required is available contiguously in the disk. Moreover, the user program need not be restarted, with the user remaining oblivious about these developments. However, the task is again time consuming.

12.3.2 Linked List Allocation

The file size generally tends to change (grow and shrink) over time. The contiguous allocation of such files results in several problems. Linked list allocation method overcomes all the problems associated with the contiguous allocation method.

In the linked list allocation method, each file is stored as a linked list of disk blocks. The disk blocks are generally scattered throughout the disk, with each disk block storing the address of the next block. The directory entry contains the file name and the address of the first and last blocks of the file (see Figure 12.4).

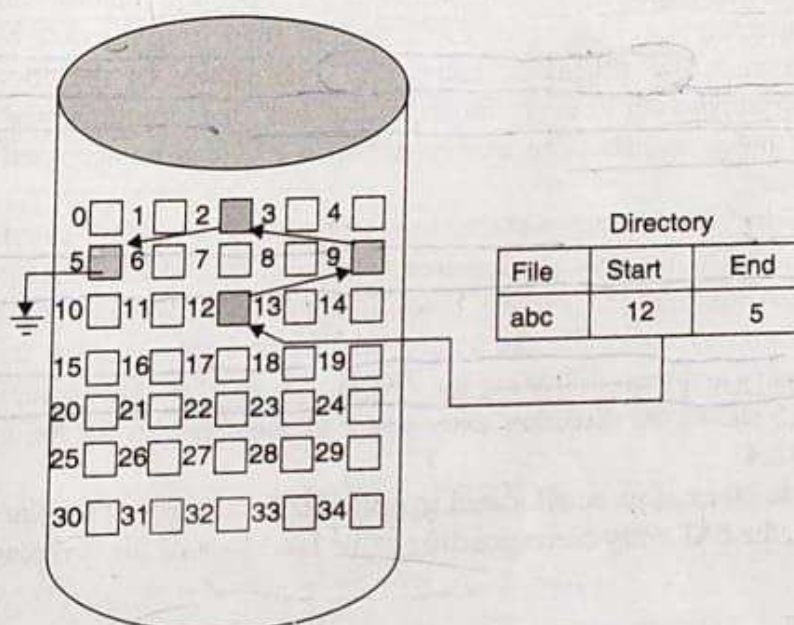


Figure 12.4: An Example of Linked List Allocation

This figure shows the linked list allocation for a file. A total of four disk blocks are allocated to the file. The directory entry indicates that the file starts at block 12. It then continues at block 9, block 2, and finally ends at block 5.

The simplicity and straightforwardness of this method makes it easy to implement. The linked list allocation results in optimum utilization of disk space, since even a single free block between the used blocks can be linked and allocated to a file. This method does not give rise to the problem of external fragmentation, so compaction is never required.

The main disadvantages of using linked list allocation are slow access speed, disk space utilization by pointers, and low-reliability of the system. As this method provides only sequential access to files, therefore, to find out the n^{th} block of a file, the search starts at the beginning of the file and follows the pointer until the n^{th} block is found. This means that there will be high average turn around time for very large files.

In linked list allocation, maintaining pointers in each block requires some disk space. The total disk space required by all the pointers in a file becomes substantial, which results in more space required by each file. The space required to store pointers can otherwise be used to store the

power of 2
slow access
Reliability.

information. To overcome this problem, contiguous blocks are grouped together as a **cluster**, and allocation to files takes place as clusters rather than blocks. Clusters allocated to a file are then linked together. Having a pointer per cluster rather than per block reduces the total space needed by all the pointers. This approach also improves the disk throughput, since fewer disk seeks are required. However, this approach may increase internal fragmentation, because having a partially full cluster wastes more space than having a partially full block.

Moreover, linked list allocation is not very reliable. Since disk blocks are linked together by pointers, a single damaged pointer may prevent us from accessing the file blocks that follow the damaged link. Some operating systems deal with this problem by creating special files for storing redundant copies of pointers. One copy of the file is placed in main memory to provide faster access to disk blocks. Other files of redundant pointers help in safer recovery.

12.3.3 Linked List Allocation using Index

In linked allocation method, problems arise because of storing pointer information within the disk blocks. To avoid these problems, a variation of linked list allocation method can be used. The idea behind this variant method is to keep pointer information apart from the disk block and store it in a separate table known as **file allocation table (FAT)** maintained by the operating system. FAT contains an entry corresponding to every block on the disk. It is implemented as an array and is indexed by the disk block number. The entries stored in FAT can be accessed in much the same manner as a linked list.

The directory entry of a file stores the address of first disk block allocated to the file; and the FAT entry corresponding to this disk block points to the next disk block allocated to that file and so on. The FAT entry corresponding to last disk block allocated to the file contains an end-of-file value (for example, -1). Thus, the entire file can be accessed by first obtaining the starting block address from the directory entry and then following the FAT entries until an entry indicating the end-of-file is found. Figure 12.5 shows the directory entry and FAT table entries for the linked list allocation depicted in Figure 12.4.

Whenever a new block is to be allocated to a file, FAT is searched for the first free block. On finding a free block, the FAT entry corresponding to the last block of file is replaced with the address

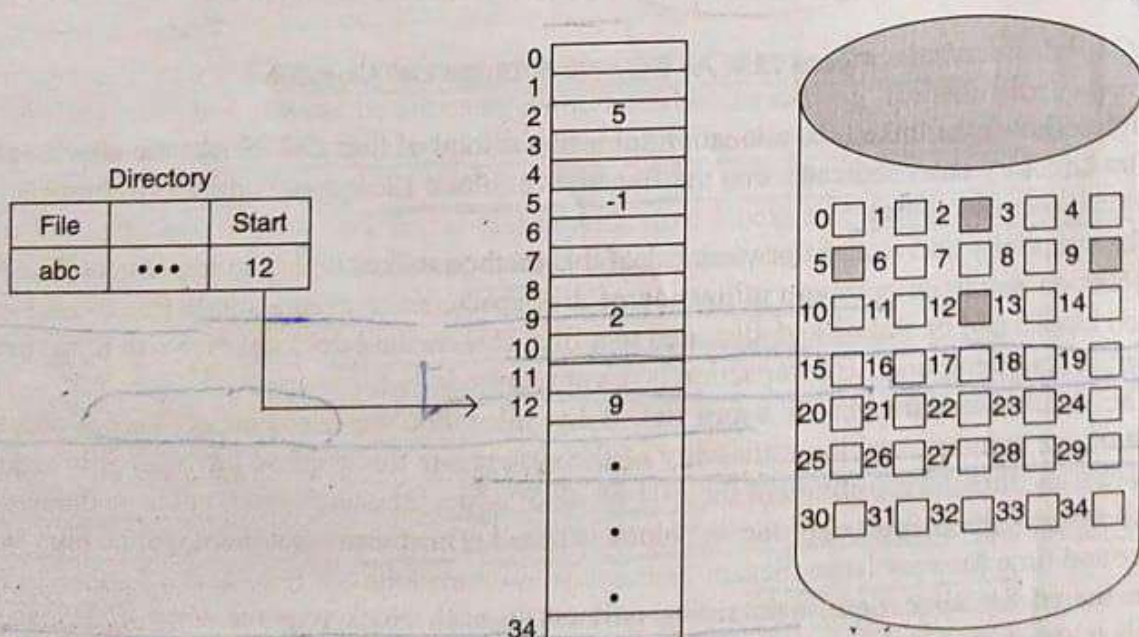


Figure 12.5: An Example of Linked List Allocation using Index

of this free block. In addition, the FAT entry of the newly allocated block is replaced with end-of-file value as now this is the last block allocated to the file.

FAT scheme is simple and efficient. However, it results in significant number of disk accesses and in turn, poor performance. That is because FAT resides on the disk and has to be accessed every time we need to obtain the address of the next disk block. Thus, to make this allocation scheme work, FAT must be kept in main memory while the file is being processed. But as the size of FAT is proportional to the size of disk, a huge amount of space is to be kept reserved in main memory. The situation becomes even worse when the disk size grows which, in turn, increases the table size.

Note: Some operating systems such as MS-DOS and Windows employ File Allocation Table scheme.

12.3.4 I-nodes

In this allocation method, each file is associated with a data structure known as **i-node (index-node)**. The i-node of a specific file contains its attributes and the disk addresses of the blocks allocated to it. This implies that the blocks of any file can be found by just having the i-node of that file in the main memory. This proves a major advantage over FAT scheme as at any instant only i-nodes of the open files need to be in main memory thereby occupying much smaller space in main memory as compared to FAT scheme. Moreover, the amount of space reserved in main memory increases as the number of open files increase, and is not linked to the size of disk.

Each i-node is capable of storing only a limited number of disk block addresses; therefore a problem arises when the size of a file increases beyond the i-node's limit. As a solution to this, indirect pointers are used in i-nodes. Initial entries of an i-node store address of the disk blocks containing the file data. The last entry of an i-node contains address of a disk block, known as **single indirect block**, which contains pointers to the additional blocks allocated to the file. (Figure 12.6).

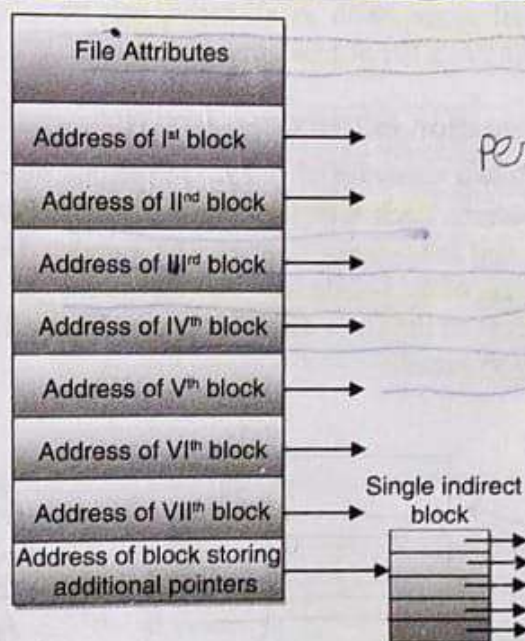


Figure 12.6: Example of an I-node

Note that to support even larger files, two or more entries in i-node can also be made to store addresses of disk blocks that contain pointers to the data blocks (disk blocks containing the file data). Sometimes, using multiple single-indirect blocks is not enough, so some operating systems (for example, UNIX) allow using double indirect blocks, to support larger file sizes. An i-node entry stores the address of a **double indirect block**, which contains the address of a disk block storing a

list of single indirect blocks. Each of these single indirect blocks contains pointers to data blocks. In case double indirect block is also not enough, triple indirect block can be used.

12.4 IMPLEMENTING DIRECTORIES

The efficiency, performance, and reliability of a file system are directly related to the directory-management and directory-allocation algorithms selected for a file system. Here, we will discuss the directory implementation in MS-DOS and UNIX.

12.4.1 Directory Implementation in MS-DOS

MS-DOS uses the linear list method for implementing directories. In this method, a directory is organized as a collection of fixed size entries, where each entry contains a (fixed-length) file name, a fixed structure to store the file attributes, and pointers to the data blocks (see Figure 12.7). All attributes of the file are stored at one place as a single directory entry and linear search is used to find a directory entry from the list of entries. It means to search out an entry in the list of entries. Each entry (starting from the first entry in the directory) is examined one by one until the desired entry is found. This is simple to program; however, the search becomes very slow with extremely large directories, which is a major disadvantage of this method.

When the user sends a request to create a new file, the directory is searched to check whether any other file has the same name. If no other file has the same name, the memory will be allocated and an entry for the same would be added at the end of the directory. To delete a file, the directory is searched for the file name and if the file is found, the space allocated to it is released. The delete operation results in free space that can be reused. To reuse this space, it can be marked with a used-unused bit, and a special name, such as all-zeros, can be assigned to it, or it can be linked to a list of free directory entries.

Directory

Itles1	attributes
language	attributes
magazine	attributes
sports	attributes
computer	attributes

12.4.2 Directory Implementation in UNIX

UNIX operating systems follow a variation of linear list method for recording the file information. Each directory entry consists of two fields: the file name and the i-node number. The i-node number contains the disk address of the i-node structure that stores the file attributes and the address of the file's data blocks (see Figure 12.8). With this approach, the size of the directory entry is very small.

Figure 12.7: Example of a Linear List Directory Entry in MS-DOS

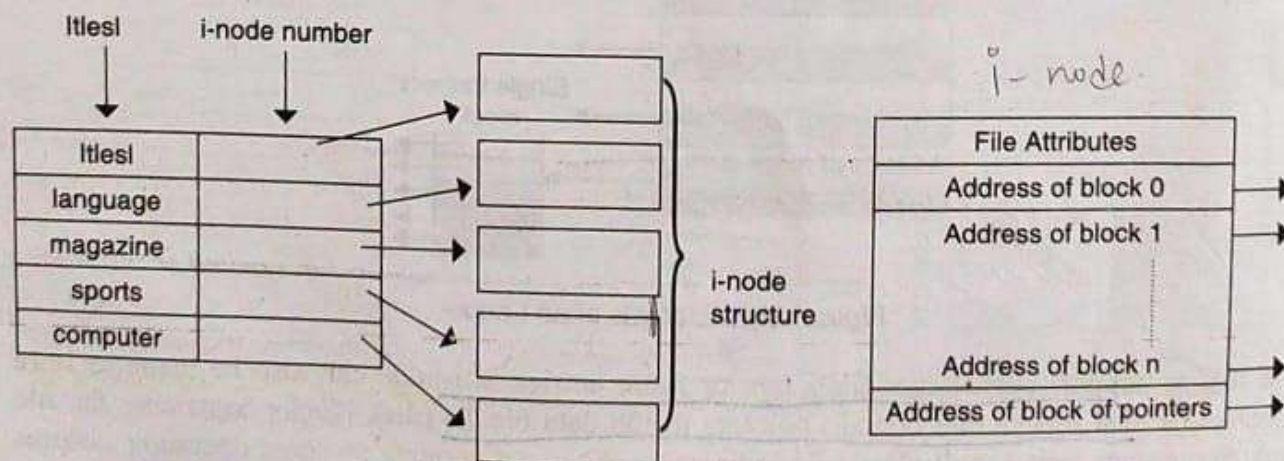


Figure 12.8: Example of a Linear List Directory Entry in UNIX